

1 Overview

The model described here is a Dual-Decoder Variational Graph Autoencoder (DualVGAE), trained jointly across a collection of graphs derived from tabular data. Each tabular dataset is converted into a graph in which rows become nodes and edges are constructed by k-nearest-neighbour (kNN) similarity in feature space. The model learns a low-dimensional latent representation of each node that captures both local graph structure and node-level feature content, and is trained to reconstruct both the graph adjacency matrix and the original node features from that representation.

The primary intended applications are unsupervised node clustering and graph generation.

2 Input Data and Graph Construction

2.1 Tabular preprocessing

Each source file (CSV, Parquet, NumPy array, or NPZ archive) is loaded into a pandas DataFrame and processed as follows:

- Drop: non-informative columns (configurable): identifier columns, timestamps, and any columns designated as leakage.
- One-hot encode categorical columns (configurable). High-cardinality categoricals should be handled upstream before this step.
- Median imputation: of missing values, applied column-wise.
- Standard scaling (zero mean, unit variance) using scikit-learn’s StandardScaler, fit independently per graph. All downstream distances and GCN operations are performed on scaled features.

The result is a dense matrix X of shape $[N, F]$, where N is the number of rows (nodes) and F is the number of features after encoding.

2.2 Graph construction by kNN

An undirected graph $G = (V, E)$ is constructed from X by connecting each node to its k most similar neighbours in feature space (default $k = 5$). Similarity is measured by cosine distance using FAISS (IndexFlatIP on L2-normalised vectors), with a fallback to scikit-learn’s `kneighbors_graph` when FAISS is unavailable.

The resulting directed kNN graph is symmetrised by taking the union of forward and reverse edges ($A \leftarrow A + A^T$), then deduplicated to produce a simple undirected graph. The graph is stored as a PyTorch Geometric Data object with node feature matrix x and adjacency encoded as a sparse edge_index tensor of shape $[2, E]$.

No edge weights are used. The graph is treated as unweighted and undirected throughout.

3 Model Architecture

The DualVGAE class extends PyTorch Geometric’s VGAE base class with a second decoder head for feature reconstruction. The full model consists of three components: a shared GCN encoder, an inner-product adjacency decoder (inherited from VGAE), and a new MLP feature decoder.

3.1 Encoder: VGAEEncoder

The encoder is a two-layer graph convolutional network implementing the Kipf & Welling (2017) spectral graph convolution (GCNConv). It maps the pair $(X, \text{edge_index})$ to a per-node posterior distribution over a latent space of dimension d .

3.1.1 Layer 1—Shared representation (conv_shared)

A single GCNConv layer maps node features from F dimensions to H dimensions (default $H = 64$), followed by a ReLU nonlinearity:

$$h_i = \rho(W_{\text{shared}} \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} X)_i,$$

where $\tilde{A} = A + I$ is the adjacency matrix with added self-loops, $\tilde{D}$ is its degree matrix, $W_{\text{shared}} \in \mathbb{R}^{FH}$ is a learned weight matrix, and $\rho(\cdot)$ is an activation function (e.g. ReLU). For each node i , this operation aggregates the (degree-normalised) features of i and all its immediate neighbours, producing a neighbourhood-aware hidden representation $h_i \in \mathbb{R}^H$. The receptive field at this layer is one hop.

3.2 Layer 2—Parallel variational heads (conv_mu, conv_logstd)

The hidden representation H is fed into two independent GCNConv layers that share no weights, each mapping $H \rightarrow d$ (default $d = 32$). This creates a Y-shaped split:

- conv_mu: produces $\mu \in \mathbb{R}^{N \times d}$, the mean of the posterior distribution $q(Z|X, A)$ for each node.
- conv_logstd: produces $\log \sigma \in \mathbb{R}^{N \times d}$, the log standard deviation. Parameterising $\log \sigma$ rather than σ directly ensures positivity without constrained optimisation, since $\sigma = \exp(\log \sigma)$ is always positive.

These two layers each perform a second round of neighbourhood aggregation, giving the encoder an effective receptive field of two hops: each node’s latent distribution is informed by nodes up to distance two in the graph.

3.3 Reparameterisation

A latent embedding z_i is sampled for each node using the reparameterisation trick:

$$z_i = \mu_i + \sigma_i \epsilon, \quad \epsilon \sim N(0, I).$$

This allows gradients to flow through the sampling operation during backpropagation: the randomness is isolated in ϵ , which has no learned parameters. The result is a matrix $Z \in \mathbb{R}^{N \times d}$ of per-node latent embeddings.

At inference time (get_embeddings), only μ is returned—the deterministic mean embedding—rather than a stochastic sample, which is standard practice for downstream tasks such as clustering.

3.3.1 Parameter count

conv_shared	$W : [F \times H] + \text{bias} : [H] \rightarrow FH + H$ parameters
conv_mu	$W : [H \times d] + \text{bias} : [d] \rightarrow Hd + d$ parameters
conv_logstd	$W : [H \times d] + \text{bias} : [d] \rightarrow Hd + d$ parameters
Total (defaults: $F = 10, H = 64, d = 32$)	$10 \times 64 + 64 + 64 \times 32 + 32 + 64 \times 32 + 32 = 5,024$ parameters

3.4 Adjacency decoder (inner product)

The adjacency decoder is the standard inner-product decoder inherited from PyTorch Geometric’s VGAE:

$$\hat{A} = \phi(ZZ^\top),$$

where ϕ is the sigmoid function and $\hat{A}_{ij} \in (0, 1)$ is the predicted probability of an edge between nodes i and j . The decoder has no learned parameters; all expressivity is in the encoder. The predicted probability is compared against the ground-truth adjacency using binary cross-entropy loss.

Because computing the full $N \times N$ matrix is infeasible for graphs with hundreds of thousands of nodes, the loss is evaluated only on a sampled set of positive edges (true edges in the training split) and an equal number of negative samples (randomly selected non-edges), following the standard VGAE training protocol.

3.5 Feature decoder (MLP)

The feature decoder is a two-layer MLP that maps the latent embedding Z back to the input feature space, producing a reconstruction $\hat{x}$:

$$\hat{x} = W_2 \rho(W_1 Z + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d \times H}$, $b_1 \in \mathbb{R}^H$, $W_2 \in \mathbb{R}^{H \times F}$, and $b_2 \in \mathbb{R}^F$ are learned parameters. The hidden dimension of the MLP is set to H (same as the encoder’s hidden dimension, default 64). The output has the same shape as the input feature matrix X , enabling direct computation of reconstruction error.

This decoder is an addition to the standard VGAE architecture and is the key structural difference in DualVGAE. It provides a direct signal to the encoder that node-level feature content should be preserved in Z , not only graph topology.

3.6 Training objective

The model is trained to minimise a weighted sum of three terms:

$$L = \alpha L_{\text{edge}} + \beta L_{\text{feat}} + \gamma L_{\text{KL}}$$

3.7 Edge reconstruction loss (L_{edge})

Binary cross-entropy between the predicted adjacency probabilities $\hat{A}$ and the ground-truth edge labels, evaluated on the training edge split (positive edges) and an equal number of randomly sampled negative edges:

$$L_{\text{edge}} = -\frac{1}{|E_{\text{train}}|} \sum_{i,j \in E_{\text{train}}} A_{ij} \log \hat{A}_{ij} + (1 - A_{ij}) \log (1 - \hat{A}_{ij})$$

This is the standard VGAE reconstruction loss. Minimising it encourages the encoder to place nodes that are connected in the graph near each other in latent space.

3.8 Feature reconstruction loss (L_{feat})

Mean squared error between the decoder’s output $\hat{x}$ and the original (scaled) node features X :

$$L_{\text{feat}} = \frac{1}{NF} \sum_i \sum_f (x_{if} - \hat{x}_{if})^2.$$

Minimising this loss encourages the encoder to retain sufficient information in Z to recover the node’s own feature values, not just its position in the graph. Because all features are standardised to unit variance before training, the MSE is expressed in units of standard deviations and is comparable across features.

3.9 KL divergence loss (L_{KL})

The Kullback-Leibler divergence between the learned posterior $q(Z \mid X, A)$ and the standard normal prior $p(Z) = N(0, I)$, normalised per node:

$$\begin{aligned} L_{\text{KL}} &= \frac{1}{N} \mathcal{D}\{q(Z \mid X, A) \parallel N(0, I)\} \\ &= \frac{1}{2N} \sum_i \sum_j (\mu_{ij}^2 + \sigma_{ij}^2 - 2 \log \sigma_{ij} - 1) \end{aligned}$$

This regularisation term penalises the encoder for producing posteriors that deviate far from the prior, which organises the latent space into a smooth, continuous manifold and makes generative sampling ($z \sim N(0, I)$) meaningful. The $1/N$ normalisation is essential when training on graphs of varying sizes, as it prevents large graphs from contributing disproportionately to the KL term.

3.10 Loss weights

The weights α, β, γ are user-configurable hyperparameters (default 1.0 each). Their primary purpose is to balance the three loss terms when they differ in magnitude at initialisation. A recommended starting procedure is to inspect the per-term losses logged at the first epoch and scale the weights so that all three terms are roughly equal before tuning for downstream task performance. Increasing α relative to β emphasises graph structure preservation; increasing β emphasises feature fidelity.

4 Optimisation and training protocol

4.1 Optimizer

All parameters (encoder GCN layers and feature decoder MLP) are trained jointly using the Adam optimiser with a fixed learning rate (default 1×10^{-3}) and PyTorch default $\beta_1 = 0.9, \beta_2 = 0.999$. No learning rate schedule or weight decay is applied by default.

4.2 Edge split

For each graph, PyTorch Geometric’s RandomLinkSplit transform holds out a fraction of edges as a validation set (default 5%) and a test set (default 5%) before training begins. The model is trained on the remaining edges only. Held-out edges are used to compute AUC and Average Precision at validation epochs, providing an unbiased estimate of edge reconstruction quality.

4.3 Multi-graph training

The model is trained sequentially across all graphs in each epoch, rather than batching nodes across graphs. This is necessary because the GCN convolution is defined with respect to each graph’s own adjacency structure, which cannot be trivially mixed across graphs. Within a single graph, PyTorch Geometric’s DataLoader is used to form mini-batches; graphs exceeding a configurable node threshold (default 300,000 nodes) are processed one at a time (batch size 1) to limit GPU memory consumption.

All graphs are loaded from disk on first encounter and cached in RAM for subsequent epochs. Given the available hardware configuration (≈ 240 GB RAM), all graphs in the dataset are expected to fit simultaneously in memory after the first epoch, eliminating repeated disk I/O.

4.4 Gradient checkpointing

When training on a CUDA device, gradient checkpointing is applied to the shared GCN layer (conv_shared). Rather than storing intermediate activations in GPU memory for use during back-propagation, the forward pass of conv_shared is recomputed during the backward pass. This trades approximately 30% additional compute for approximately 40% reduction in GPU activation memory, and is particularly beneficial for the largest graphs.

5 Model outputs

5.1 Node embeddings

After training, the encoder is applied to each graph in inference mode (no sampling; $Z = \mu$) to produce a matrix of node embeddings of shape $[N, d]$. Embeddings are exported per graph as CSV files (one row per node, d columns z_0 through z_{d-1}). These embeddings are the primary output for downstream tasks.

5.2 Generated graphs

New graphs can be generated by sampling latent vectors $z \sim N(0, I)$ and passing them through the inner-product decoder. The result is a probabilistic adjacency matrix $\hat{A}$; thresholding at a chosen probability (default 0.5) gives a binary adjacency matrix. Generated graphs have no associated node features unless a separate feature generation step is added.

5.3 Training loss history

All four loss values (total, edge, feature, KL) are recorded per epoch and written to a CSV file at the end of training. This breakdown facilitates post-hoc diagnosis of imbalanced loss terms and tuning of α, β, γ .

6 Evaluation

6.1 Adjacency reconstruction

Edge reconstruction quality is measured using two metrics computed on the held-out validation edges and an equal number of randomly sampled negative edges:

- AUC (Area Under the ROC Curve): measures the model’s ability to rank true edges above non-edges. A random model scores 0.5; a well-trained VGAE typically achieves 0.85 – 0.95 on structured graph data.
- Average Precision (AP): the area under the precision-recall curve, which is more informative than AUC when the graph is sparse (far more non-edges than edges), as is typical of kNN-constructed graphs.

6.2 Feature reconstruction

For each graph and each feature dimension, the following metrics are computed between the original scaled features X and the decoder output $\hat{x}$:

Metric	Formula	Interpretation
MSE	$\frac{1}{N} \ x - \hat{x}\ ^2$	Mean squared error in units of standard deviations (features are standardised). Lower is better.
MAE	$\frac{1}{N} \sum_i x_i - \hat{x}_i $	Mean absolute error; less sensitive to outliers than MSE.
R^2	$1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$	Fraction of variance explained by the decoder.

Results are aggregated across graphs and summarised per feature dimension, enabling identification of which features are well-preserved in the latent space and which are discarded. Low R^2 for a feature is not necessarily a failure: it may indicate that the feature is idiosyncratic to individual nodes rather than correlated with local graph structure, and is therefore not recoverable from a neighbourhood-aware embedding.

6.3 Embedding quality

Node embeddings are evaluated for clustering quality using:

- Silhouette score $\in [-1, 1]$: measures cohesion within clusters relative to separation between clusters. Values above 0.3 indicate meaningful structure; above 0.5 is strong. Computed on a random subsample of nodes ($\leq 10,000$) due to $\mathcal{O}(N^2)$ complexity.
- Davies-Bouldin index: ratio of within-cluster scatter to between-cluster separation; lower is better. Computed on the full embedding as it scales as $\mathcal{O}(Nk)$.
- k sweep: both metrics are evaluated across a range of k values for k -means clustering, with the optimal k identified as the peak in silhouette score and the elbow in inertia.

Prior coverage of the latent space is assessed by verifying that the mean and standard deviation of Z across all nodes and all graphs are close to 0 and 1 respectively, matching the $N(0, I)$ prior from which generated graphs are sampled.

7 Tuning parameters

k	Number of nearest neighbours per node in kNN graph construction. Default: 5. Increase for denser graphs; decrease if the model overfits to local structure.
hidden_channels (H)	Dimensionality of the shared GCN hidden layer and the MLP feature decoder’s hidden layer. Default: 64.
latent_dim (d)	Dimensionality of the latent space. Default: 32. Increase if clustering results are indistinct; decreasing improves generation regularity.
loss_alpha (α)	Weight on the edge reconstruction BCE loss. Default: 1.0.
loss_beta (β)	Weight on the feature reconstruction MSE loss. Default: 1.0.
loss_gamma (γ)	Weight on the KL divergence term. Default: 1.0. Increasing γ produces a more organised latent space at the cost of reconstruction fidelity.
epochs	Number of full passes through the graph collection. Default: 200.
lr	Adam learning rate. Default: 1×10^{-3} .
batch_size	Number of graphs per DataLoader batch (for graphs below large_graph_threshold). Default: 4.
large_graph_threshold	Graphs with more nodes than this value are processed with batch_size=1 regardless of the batch_size setting. Default: 300,000.
val_fraction	Fraction of edges held out for validation. Default: 0.05.
test_fraction	Fraction of edges held out for testing. Default: 0.05.

8 Dependencies

PyTorch	Tensor operations, autograd, Adam optimiser
PyTorch Geometric	GCNConv, VGAE base class, Data/DataLoader, RandomLinkSplit
FAISS	Approximate kNN graph construction (CPU or GPU)
scikit-learn	StandardScaler, KMeans, silhouette_score, davies_bouldin_score; kneighbors_graph fallback
pandas/NumPy	Tabular data loading, preprocessing, CSV output
psutil	Runtime memory availability reporting